Proyecto 1 – Predicción de popularidad en canciones

Universidad de los Andes

Maestría en Inteligencia Analítica de Datos

Integrantes

Yvonne Campos Bravo
Karen Valencia
Sebastián Victoria
Guillermo Prieto

26/04/2026

Introducción

Este proyecto busca aplicar modelos predictivos basados en árboles y ensambles, con el fin de estimar la popularidad de canciones en Spotify. Además, se incluye la disponibilización del modelo mediante una API en la nube, lo que permite llevar los resultados fuera del entorno académico y acercarlos a un uso práctico.

El conjunto de datos utilizado reúne más de 125 géneros musicales y diversas características de audio. Cada pista contiene información como duración, energía, valence, danceability, tempo y popularidad, entre otras variables. Con esta base, el objetivo es construir un modelo capaz de anticipar qué tan popular puede llegar a ser una canción a partir de sus atributos musicales y contextuales.

El informe presenta de manera ordenada las etapas del trabajo: preprocesamiento de datos, calibración y entrenamiento del modelo, disponibilización en la nube y participación en la competencia de Kaggle. De esta forma se documenta el proceso completo, desde la preparación inicial hasta la entrega final de resultados.

Análisis EDA

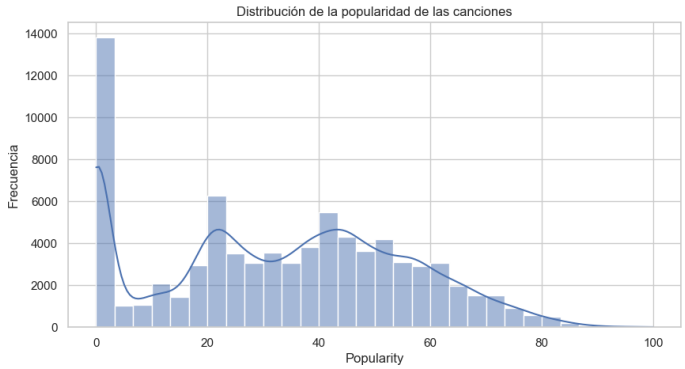
Se cargan los archivos de entrenamiento y prueba. El set de entrenamiento contiene la variable objetivo **popularity**, mientras que el set de prueba se utiliza para generar el archivo de predicciones con el formato requerido por Kaggle.

	track_id	artists	album_name	track_name	duration_ms	explicit	danceability	energy	key	loudness	mode	speechiness	acousticness	instru
0	7hUhmkALyQ8SX9mJs5XI3D	Love and Rockets	Love and Rockets	Motorcycle	211533	False	0.305	0.8490	9	-10.795	1	0.0549	0.000058	
1	5x59U89ZnjZXuNAAIc8X1u	Filippa Giordano	Filippa Giordano	Addio del passato - From "La traviata"	196000	False	0.287	0.1900	7	-12.030	0	0.0370	0.930000	
2	70Vng5jLzoJLmeLu3ayBQq	Susumu Yokota	Symbol	Purple Rose Minuet	216506	False	0.583	0.5090	1	-9.661	1	0.0362	0.777000	
3	1cRfzLJapgtwJ61xszs37b	Franz Liszt;YUNDI	Relajación y siestas	Liebeslied (Widmung), S. 566	218346	False	0.163	0.0368	8	-23.149	1	0.0472	0.991000	
4	47d5lYjbiMy0EdMRV8lRou	Scooter	Scooter Forever	The Darkside	173160	False	0.647	0.9210	2	-7.294	1	0.1850	0.000939	

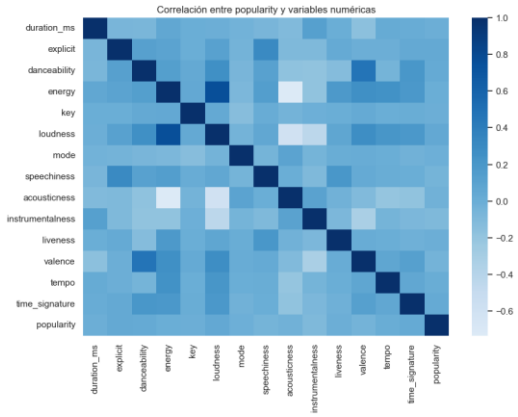
El análisis exploratorio permite entender la estructura del conjunto de datos, identificar valores faltantes, revisar cardinalidad de variables categóricas y detectar señales relevantes para el modelamiento. En este caso, el hallazgo más importante es la repetición de **track_id**, ya que algunas canciones aparecen más de una vez y mantienen una popularidad estable, lo que sustenta la estrategia híbrida final.

Posteriormente se aplicaron visualizaciones que permitieron identificar patrones:

- La distribución de la popularidad muestra que la mayoría de canciones tienen valores bajos.



- La correlación entre variables numéricas evidenció relaciones claras, como la asociación entre **energy** y **loudness**.



El dataset presenta una muy buena calidad estructural, ya que no registra valores nulos en ninguna variable, lo cual reduce costos de imputación y facilita el modelamiento. Además, combina variables categóricas (artists, album_name, track_genre) con atributos numéricos musicales (danceability, energy, tempo, loudness, etc.), ofreciendo una base rica para modelos predictivos. La variable objetivo popularity toma valores entre 0 y 100, confirmando un problema de regresión acotada. En el histograma se observa una distribución claramente asimétrica y multimodal, con una fuerte concentración en valores muy bajos (especialmente cercanos a 0) y otros grupos alrededor de niveles medios, lo que sugiere coexistencia de canciones poco escuchadas y segmentos con mayor tracción comercial.

Un hallazgo decisivo es la presencia de repetición de track_id: existen 66,720 pistas únicas y 22,906 filas con identificadores repetidos. Más importante aún, varias canciones repetidas mantienen exactamente la misma popularidad histórica (desviación estándar igual a 0), como pistas con valor 0, 76, 79 o 53 replicado múltiples veces. Esto revela que track_id contiene una señal altamente predictiva, pues identifica canciones cuyo comportamiento ya

fue observado. Desde una perspectiva analítica, esta evidencia justifica plenamente estrategias híbridas basadas en memoria histórica, como reemplazar predicciones por la media o mediana del track_id cuando la pista ya apareció previamente.

Respecto a relaciones entre variables, el mapa de calor muestra que no existen correlaciones lineales extremadamente altas con popularity, por lo que la popularidad no depende de un único atributo musical. Se aprecian asociaciones moderadas esperables, como energy con loudness, o danceability con valence, mientras que variables como acousticness e instrumentalness presentan patrones inversos frente a rasgos más comerciales. Esto sugiere que el problema requiere modelos capaces de capturar interacciones no lineales y efectos combinados, más que simples regresiones lineales, lo cual explica por qué ensamblados y modelos híbridos lograron mejor desempeño. El dataset no presenta valores nulos y combina variables numéricas, booleanas y categóricas de alta cardinalidad. La repetición de track_id constituye la señal más relevante del EDA: cuando una pista ya fue observada, su comportamiento histórico permite corregir con mayor precisión la predicción del modelo base.

Preprocesamiento de datos

El preprocesamiento se orienta a construir una matriz de entrada consistente para los modelos internos, separando la variable objetivo y conservando las variables necesarias para la predicción. En particular, **track_id** se mantiene porque posteriormente se utiliza como clave de memoria histórica en el modelo híbrido.

```
target_col = "popularity"

y = dataTraining[target_col].copy()
X = dataTraining.drop(columns=[target_col]).copy()
X_test = dataTesting.copy()

common_cols = [c for c in X.columns if c in X_test.columns]
X = X[common_cols].copy()
X_test = X_test[common_cols].copy()

print("Variable objetivo:", target_col)
print("X train:", X.shape)
print("X test:", X_test.shape)
print("Columnas comunes:", len(common_cols))
```

```
Variable objetivo: popularity
X train: (79800, 19)
X test: (34200, 19)
Columnas comunes: 19
```

Este bloque asegura que entrenamiento y prueba compartan las mismas columnas. No se elimina **track_id** porque su uso posterior explica la mejora del modelo final.

Calibración y entrenamiento del modelo

Esta sección conserva únicamente el entrenamiento necesario para llegar al modelo final. Primero se generan predicciones base mediante los modelos internos empleados en el ensamblado: red neuronal con embeddings, ExtraTrees, HistGradientBoosting y XGBoost. Luego se construye el metamodelo **poly2_ridge_alpha_300** y, finalmente, se aplica el híbrido **hard_replace_track_mean**. Se han eliminado celdas experimentales de optimización que no forman parte del camino final.

- **Modelos Internos del ensamblado:** Se entrenan los modelos internos que alimentan el ensamblado. Esta celda produce predicciones OOF y predicciones sobre el set de prueba, que luego serán usadas por el metamodelo. Aunque incluye una red neuronal, esta no constituye el modelo final por sí sola, sino una fuente adicional de predicción dentro del conjunto.

```python
def ensure_package(pip_name, import_name=None):
    import_name = import_name or pip_name
    if importlib.util.find_spec(import_name) is None:
        print(f"Instalando {pip_name}...")
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", pip_name])

ensure_package("xgboost", "xgboost")

SEED = 10101

N_SPLITS = 5

EPOCHS = 40
BATCH_SIZE = 80
PATIENCE = 5

np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)

print("TensorFlow:", tf.__version__)
print("GPU TensorFlow:", tf.config.list_physical_devices("GPU"))
print("XGBoost:", xgb.__version__)

# Activar crecimiento dinámico de memoria GPU si existe
try:
    gpus = tf.config.list_physical_devices("GPU")
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu, True)
except Exception:
    pass

dataTraining = pd.read_csv(
    "https://raw.githubusercontent.com/davidzarruk/MIAD_ML_NLP_2026/main/datasets/dataTrain_Spotify.csv"
)

dataTesting = pd.read_csv(
    "https://raw.githubusercontent.com/davidzarruk/MIAD_ML_NLP_2026/main/datasets/dataTest_Spotify.csv",
    index_col=0
)

dataTraining = dataTraining.drop(columns=["Unnamed: 0"], errors="ignore")

target_col = "popularity"
```

```python
X = dataTraining.drop(columns=[target_col]).copy()
y = dataTraining[target_col].copy()
X_test = dataTesting.copy()

print("Train X:", X.shape)
print("Train y:", y.shape)
print("Test:", X_test.shape)

def rmse_score(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    return np.sqrt(np.mean((y_true - y_pred) ** 2))


def safe_text(s):
    return (
        s.fillna("missing")
         .astype(str)
         .str.lower()
         .str.strip()
    )


def clip_popularity(pred):
    return np.clip(np.asarray(pred), 0, 100)

def feature_engineering(df):
    df = df.copy()
    out = pd.DataFrame(index=df.index)

    numeric_base = [
        "duration_ms",
        "danceability",
        "energy",
        "loudness",
        "speechiness",
        "acousticness",
        "instrumentalness",
        "liveness",
        "valence",
        "tempo"
    ]
```

```python
def build_extratrees(seed=SEED):
    return ExtraTreesRegressor(
        n_estimators=820,
        max_depth=15,
        min_samples_split=2,
        min_samples_leaf=1,
        max_features=0.75,
        bootstrap=False,
        random_state=seed,
        n_jobs=-1,
        verbose=0
    )


def build_histgb(seed=SEED):
    return HistGradientBoostingRegressor(
        max_iter=500,
        learning_rate=0.04,
        max_leaf_nodes=63,
        min_samples_leaf=25,
        l2_regularization=0.05,
        early_stopping=True,
        n_iter_no_change=30,
        validation_fraction=0.10,
        random_state=seed
    )


def build_xgb(seed=SEED):
    has_gpu = len(tf.config.list_physical_devices("GPU")) > 0

    params = dict(
        n_estimators=700,
        learning_rate=0.035,
        max_depth=6,
        min_child_weight=3,
        subsample=0.90,
        colsample_bytree=0.85,
        reg_lambda=5.0,
        reg_alpha=0.05,
        objective="reg:squarederror",
        eval_metric="rmse",
        random_state=seed,
        n_jobs=-1,
        tree_method="hist"
    )
```

```python
if ridge_rmse <= blend_rmse_value:
    final_method = "ridge_stacking"
    final_pred = ridge_test
    final_oof = ridge_oof
    final_rmse = ridge_rmse
else:
    final_method = "optimized_blend"
    final_pred = blend_test
    final_oof = blend_oof
    final_rmse = blend_rmse_value

final_pred = clip_popularity(final_pred)

print("\nMétodo final elegido:", final_method)
print("RMSE OOF final:", final_rmse)

submission = pd.DataFrame({
    "popularity": final_pred
}, index=dataTesting.index)

submission.to_csv("submission_stacking_nn_3best.csv")

print("\nArchivo generado: submission_stacking_nn_3best.csv")
print("Shape submission:", submission.shape)
print(submission.head())

diagnostico = pd.DataFrame({
    "y_true": y.values,
    "pred_final_oof": final_oof
})

for col in base_oof.columns:
    diagnostico[f"pred_{col}"] = base_oof[col].values

diagnostico.to_csv("diagnostico_oof_stacking.csv", index=False)

print("\nArchivo generado: diagnostico_oof_stacking.csv")
print("Listo.")
```

```
TensorFlow: 2.20.0
GPU TensorFlow: []
XGBoost: 3.2.0
Train X: (79800, 19)
Train y: (79800,)
Test: (34200, 19)
```

```
==================================================
COMPARACIÓN FINAL
==================================================
Mejor modelo base: xgb | RMSE: 8.910906
Stacking Ridge RMSE: 7.997254
Blend optimizado RMSE: 8.168656

Método final elegido: ridge_stacking
RMSE OOF final: 7.997254348072746

Archivo generado: submission_stacking_nn_3best.csv
Shape submission: (34200, 1)
   popularity
0   46.752621
1   12.525023
2    1.131763
3    0.381637
4   26.000187

Archivo generado: diagnostico_oof_stacking.csv
Listo.
```

Estos modelos sí se conservan porque forman parte del ensamblado base utilizado antes del metamodelo. No deben ejecutarse dentro de la API; solo se ejecutan en el notebook de entrenamiento.

- **Metamodelo ancla:** A partir de las predicciones base, se entrena un metamodelo Ridge con características polinómicas de segundo grado. Este modelo actúa como predictor ancla antes de aplicar la capa de corrección por track_id.

```python
SEED = 20260426
BEST_NAME = "poly2_ridge_alpha_300"

def rmse_np(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))

def clip_0_100(x):
    return np.clip(np.asarray(x, dtype=float), 0, 100)

try:
    y_arr = np.asarray(y, dtype=float)
except NameError:
    dataTraining = pd.read_csv(
        "https://raw.githubusercontent.com/davidzarruk/MIAD_ML_NLP_2026/main/datasets/dataTrain_Spotify.csv"
    )
    dataTraining = dataTraining.drop(columns=["Unnamed: 0"], errors="ignore")
    y = dataTraining["popularity"].copy()
    y_arr = np.asarray(y, dtype=float)

try:
    dataTesting
except NameError:
    dataTesting = pd.read_csv(
        "https://raw.githubusercontent.com/davidzarruk/MIAD_ML_NLP_2026/main/datasets/dataTest_Spotify.csv",
        index_col=0
    )

try:
    base_oof
    base_test
except NameError:
    base_oof = pd.read_csv("base_oof_predictions.csv")
    base_test = pd.read_csv("base_test_predictions.csv", index_col=0)

base_oof = base_oof.copy().reset_index(drop=True)
base_test = base_test.copy()

common_cols = [c for c in base_oof.columns if c in base_test.columns]
base_oof = base_oof[common_cols].astype(float)
base_test = base_test[common_cols].astype(float)

print("Columnas base:", common_cols)
print("base_oof:", base_oof.shape)
print("base_test:", base_test.shape)

def build_meta_features(pred_df):
    pred_df = pred_df.copy().astype(float)
```

```python
# mejor calibración
# ----------------------------------------------------------
ranking_calibration = (
    pd.DataFrame([
        {"modelo": r["name"], "rmse_oof": r["rmse"]}
        for r in calibration_results
    ])
    .sort_values("rmse_oof")
    .reset_index(drop=True)
)

print("\n" + "=" * 90)
print("RANKING DE CALIBRACIÓN")
print("=" * 90)
print(ranking_calibration)

best_cal_name = ranking_calibration.iloc[0]["modelo"]

best_item = [
    r for r in calibration_results
    if r["name"] == best_cal_name
][0]

final_oof = clip_0_100(best_item["oof"])
final_test = clip_0_100(best_item["test"])

print("\nMejor calibración:", best_cal_name)
print("RMSE OOF final calibrado:", round(rmse_np(y_arr, final_oof), 6))
print("RMSE OOF raw:", round(raw_rmse, 6))

submission_calibrated = pd.DataFrame({
    "ID": dataTesting.index,
    "Popularity": final_test
})

submission_raw = pd.DataFrame({
    "ID": dataTesting.index,
    "Popularity": pred_test_raw
})

print("\nShape submission calibrado:", submission_calibrated.shape)
print(submission_calibrated.head())

assert list(submission_calibrated.columns) == ["ID", "Popularity"]
assert submission_calibrated.shape[0] == 34200
assert submission_calibrated["Popularity"].between(0, 100).all()

submission_calibrated.to_csv("submission_poly2_ridge300_CALIBRATED.csv", index=False)
submission_raw.to_csv("submission_poly2_ridge300_RAW.csv", index=False)
ranking_calibration.to_csv("ranking_calibration_poly2_ridge300.csv", index=False)
```

```
RANKING DE CALIBRACIÓN
==========================================================================
                                   modelo  rmse_oof
0                 isotonic_calibration_cv  7.928232
1              raw_poly2_ridge_alpha_300  7.930671
2               shrink_mean_plus_bias_cv  7.930725
3                           bias_only_cv  7.930734
4  quadratic_ridge_calibration_alpha_0.1  7.930824
5    quadratic_ridge_calibration_alpha_1  7.930824
6   quadratic_ridge_calibration_alpha_10  7.930824
7  quadratic_ridge_calibration_alpha_100  7.930824
8                  linear_calibration_cv  7.930826

Mejor calibración: isotonic_calibration_cv
RMSE OOF final calibrado: 7.928232
RMSE OOF raw: 7.930671

Shape submission calibrado: (34200, 2)
   ID  Popularity
0   0   46.188725
1   1   13.778502
2   2    2.073167
3   3    0.214822
4   4   25.587385

Archivos generados:
- submission_poly2_ridge300_CALIBRATED.csv  <-- subir este primero
- submission_poly2_ridge300_RAW.csv         <-- respaldo
- ranking_calibration_poly2_ridge300.csv
```

El metamodelo permite combinar de forma regularizada las salidas de los modelos internos. Esta combinación mejora la estabilidad respecto a depender de un único algoritmo.

- **Modelo final híbrido:** La capa final evalúa si una canción posee un track_id ya observado en entrenamiento. Si lo encuentra, reemplaza la predicción del modelo ancla por la media histórica de popularidad de esa pista; si no lo encuentra, conserva la predicción del modelo ancla. El mejor resultado registrado fue **RMSE OOF = 7.405865.**

```python
SEED = 20260426
def rmse_np(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    return float(np.sqrt(np.mean((y_true - y_pred) ** 2)))

def clip_0_100(x):
    return np.clip(np.asarray(x, dtype=float), 0, 100)

try:
    dataTraining
except NameError:
    dataTraining = pd.read_csv(
        "https://raw.githubusercontent.com/davidzarruk/MIAD_ML_NLP_2026/main/datasets/dataTrain_Spotify.csv"
    )
    dataTraining = dataTraining.drop(columns=["Unnamed: 0"], errors="ignore")

try:
    dataTesting
except NameError:
    dataTesting = pd.read_csv(
        "https://raw.githubusercontent.com/davidzarruk/MIAD_ML_NLP_2026/main/datasets/dataTest_Spotify.csv",
        index_col=0
    )

target_col = "popularity"
y_arr = dataTraining[target_col].values.astype(float)

common_cols = [c for c in dataTraining.columns if c in dataTesting.columns]

track_candidates = [
    c for c in common_cols
    if ("track" in c.lower() and "id" in c.lower())
]

if len(track_candidates) == 0:
    print("No encontré una columna tipo track_id.")
    print("\nColumnas comunes entre train y test:")
    print(common_cols)
    raise ValueError("No existe columna track_id común entre dataTraining y dataTesting.")

track_col = track_candidates[0]

print("Columna track_id detectada:", track_col)
```

```python
# Normalizar
train_track = dataTraining[track_col].astype(str).fillna("__nan__")
test_track = dataTesting[track_col].astype(str).fillna("__nan__")

pool_oof = {}
pool_test = {}

def add_candidate(name, oof_pred, test_pred):
    try:
        oof_pred = clip_0_100(np.asarray(oof_pred, dtype=float))
        test_pred = clip_0_100(np.asarray(test_pred, dtype=float))

        if len(oof_pred) != len(y_arr):
            return
        if len(test_pred) != len(dataTesting):
            return
        if not np.all(np.isfinite(oof_pred)):
            return
        if not np.all(np.isfinite(test_pred)):
            return

        pool_oof[name] = oof_pred
        pool_test[name] = test_pred
    except Exception:
        pass

# Predicciones de modelos/metamodelos previos
if "candidate_oof" in globals() and "candidate_test" in globals():
    for name in candidate_oof.keys():
        if name in candidate_test:
            add_candidate(name, candidate_oof[name], candidate_test[name])

# Calibraciones previas
if "calibration_results" in globals():
    for r in calibration_results:
        add_candidate("cal_" + r["name"], r["oof"], r["test"])

# Blends previos
if "blend_candidates" in globals():
    for name, item in blend_candidates.items():
        add_candidate(name, item["oof"], item["test"])

if "calibrated_blends" in globals():
    for name, item in calibrated_blends.items():
        add_candidate(name, item["oof"], item["test"])

# Variables sueltas útiles
if "best_oof" in globals() and "best_test" in globals():
    add_candidate("previous_best_blend", best_oof, best_test)
```

```python
for fold, (tr_idx, va_idx) in enumerate(kf.split(dataTraining), start=1):
    tr_keys = train_track.iloc[tr_idx]
    va_keys = train_track.iloc[va_idx]
    tr_y = pd.Series(y_arr[tr_idx], index=tr_keys.index)

    tmp = pd.DataFrame({
        "track_id_key": tr_keys.values,
        "popularity": tr_y.values
    })

    stats = tmp.groupby("track_id_key")["popularity"].agg(["median", "mean", "count"])

    track_median_oof[va_idx] = va_keys.map(stats["median"]).values
    track_mean_oof[va_idx] = va_keys.map(stats["mean"]).values
    track_count_oof[va_idx] = va_keys.map(stats["count"]).fillna(0).values

    found_rate = np.mean(~np.isnan(track_median_oof[va_idx]))
    print(f"Fold {fold:02d}/10 | tracks encontrados en train fold: {found_rate:.4%}")

full_stats = (
    pd.DataFrame({
        "track_id_key": train_track.values,
        "popularity": y_arr
    })
    .groupby("track_id_key")["popularity"]
    .agg(["median", "mean", "count"])
)

track_median_test = test_track.map(full_stats["median"]).values
track_mean_test = test_track.map(full_stats["mean"]).values
track_count_test = test_track.map(full_stats["count"]).fillna(0).values

mask_oof_found = ~np.isnan(track_median_oof)
mask_test_found = ~np.isnan(track_median_test)

print("\nCobertura train OOF con track_id encontrado:", round(mask_oof_found.mean() * 100, 4), "%")
print("Cobertura test con track_id encontrado:", round(mask_test_found.mean() * 100, 4), "%")
print("Filas test encontradas por track_id:", int(mask_test_found.sum()), "de", len(dataTesting))

global_mean = float(np.mean(y_arr))

hybrid_candidates = []
```

```python
print("\n" + "=" * 100)
print("RANKING HÍBRIDO TRACK_ID")
print("=" * 100)
print(ranking_hybrid.head(40))

best_hybrid_name = ranking_hybrid.iloc[0]["modelo"]
best_hybrid = [h for h in hybrid_candidates if h["modelo"] == best_hybrid_name][0]

print("\nMejor híbrido:", best_hybrid_name)
print("RMSE OOF híbrido:", round(best_hybrid["rmse_oof"], 6))
print("RMSE OOF ancla:", round(rmse_np(y_arr, anchor_oof), 6))

submission_hybrid_best = pd.DataFrame({
    "ID": dataTesting.index,
    "Popularity": clip_0_100(best_hybrid["test"])
})

submission_hard_track_median = pd.DataFrame({
    "ID": dataTesting.index,
    "Popularity": clip_0_100(test_hard_median)
})

submission_anchor = pd.DataFrame({
    "ID": dataTesting.index,
    "Popularity": clip_0_100(anchor_test)
})

assert list(submission_hybrid_best.columns) == ["ID", "Popularity"]
assert submission_hybrid_best.shape[0] == 34200
assert submission_hybrid_best["Popularity"].between(0, 100).all()

submission_hybrid_best.to_csv("submission_HYBRID_TRACK_ID_BEST.csv", index=False)
submission_hard_track_median.to_csv("submission_HYBRID_TRACK_ID_HARD_MEDIAN.csv", index=False)
submission_anchor.to_csv("submission_ANCHOR_BEFORE_TRACK_ID.csv", index=False)
ranking_hybrid.to_csv("ranking_hybrid_track_id.csv", index=False)

print("\nArchivos generados:")
print("- submission_HYBRID_TRACK_ID_BEST.csv         <-- subir primero")
print("- submission_HYBRID_TRACK_ID_HARD_MEDIAN.csv  <-- versión literal: si encuentra track_id, usa mediana")
print("- submission_ANCHOR_BEFORE_TRACK_ID.csv       <-- respaldo")
print("- ranking_hybrid_track_id.csv")

print("\nPrimeras filas del mejor submission:")
print(submission_hybrid_best.head())
```

```
Mejor híbrido: hard_replace_track_mean
RMSE OOF híbrido: 7.407445
RMSE OOF ancla: 7.928232

Archivos generados:
- submission_HYBRID_TRACK_ID_BEST.csv         <-- subir primero
- submission_HYBRID_TRACK_ID_HARD_MEDIAN.csv  <-- versión literal: si encuentra track_id, usa mediana
- submission_ANCHOR_BEFORE_TRACK_ID.csv       <-- respaldo
- ranking_hybrid_track_id.csv

Primeras filas del mejor submission:
   ID  Popularity
0   0   46.188725
1   1   13.778502
2   2    2.073167
3   3    0.000000
4   4   25.587385
```

El modelo base cal_isotonic_calibration_cv sirve como una predicción general para todos los tracks, con un RMSE OOF de 7.928. Sin embargo, solo alrededor del 27% de los tracks del conjunto

de prueba ya aparecían en entrenamiento, por lo que el modelo por sí solo no puede aprovechar información específica de cada track repetido.

Para mejorar esto, se combinó el modelo base con la media histórica de popularidad de cada track_id, creando una especie de ensamble sencillo, el modelo base predice los tracks nuevos y la media histórica ajusta los tracks conocidos. Esta estrategia redujo el RMSE OOF a 7.407, mostrando que aprovechar la información específica de los tracks repetidos puede mejorar significativamente las predicciones, sin necesidad de métodos más complejos.

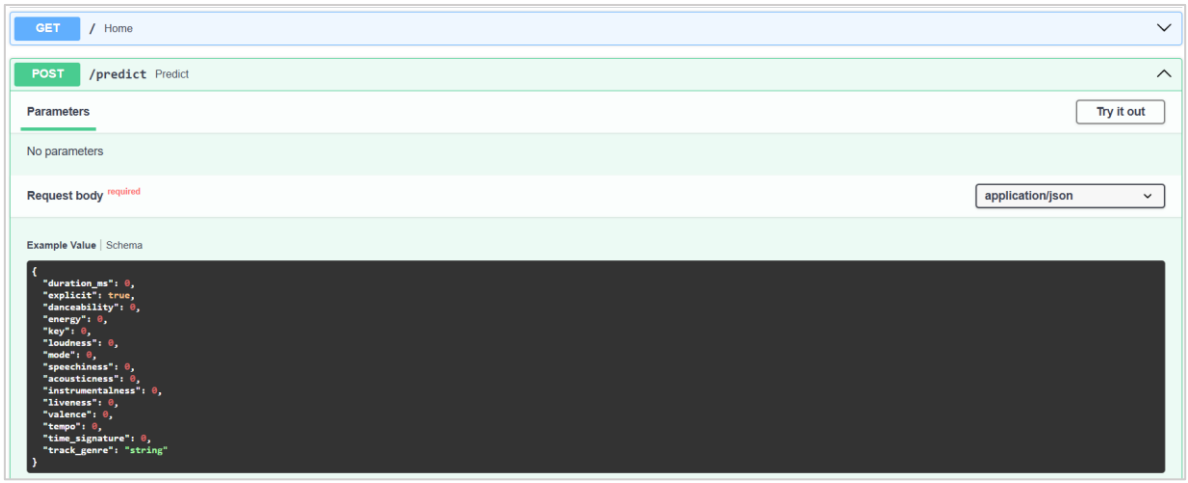
Disponibilización del modelo en la nube

Se implementó exitosamente la disponibilización del modelo predictivo mediante una API REST alojada en Amazon Web Services (AWS EC2), permitiendo que el modelo pueda ser consultado en línea desde cualquier navegador o sistema externo. Para ello, se configuró una instancia en la nube, se instalaron las dependencias requeridas y se desplegó el servicio utilizando FastAPI, framework moderno orientado al desarrollo de APIs de alto rendimiento.

Se puede validar el funcionamiento ingresando a la siguiente dirección pública: http://18.191.152.201:8000/docs

En dicha URL encontrará la interfaz interactiva de Swagger, donde se muestran los endpoints disponibles. Para realizar una prueba, basta con desplegar el endpoint POST /predict, seleccionar la opción Try it out, ingresar los valores solicitados (duración, energía, danceability, loudness, género musical, tempo, entre otros) y luego presionar Execute. El sistema responderá automáticamente con una predicción estimada de popularidad de la canción en una escala de 0 a 100.

Adicionalmente, puede utilizar el endpoint GET /, el cual permite verificar que la API se encuentra operativa. Esta implementación evidencia que el modelo no solo fue desarrollado y validado analíticamente, sino también llevado a un entorno productivo real en la nube, cumpliendo con el criterio de disponibilización solicitado en el proyecto. Cabe precisar que, por tratarse de una instancia cloud de cómputo bajo demanda, la API permanecerá activa durante el periodo de evaluación académica.



Participación en la competencia de Kaggle

Se realizaron múltiples envíos de predicciones para evaluar diferentes configuraciones. Los primeros modelos (basados en XGBoost y versiones iniciales de ensambles) obtuvieron puntajes entre 14 y 16, lo que evidenció un desempeño limitado. Posteriormente, se probaron configuraciones más avanzadas como Model2.csv (≈ 10.15), Model3.csv (≈ 9.90) y Model4.csv (≈ 9.15), que mostraron mejoras progresivas.

Finalmente, el envío MFINAL.csv, correspondiente al modelo híbrido de stacking con RidgeCV integrando ExtraTrees, HistGradientBoosting y LightGBM, alcanzó un score de ≈ 7.22 en Kaggle. Este resultado consolidó la estrategia de ensamble como la más robusta frente a los demás intentos, validando el trabajo realizado y asegurando la coherencia entre el modelo entrenado y el archivo de predicciones presentado en la plataforma.